

1 Plotting

```
ggplot2::ggplot(df, aes(X1, Y))+  
  geom_point() +  
  geom_smooth(method="lm")  
  
ggplot2::ggplot(df, aes(X1, Y))+  
  geom_line()
```

```
ggplot2::ggplot(df, aes(C1, Y))+  
  geom_boxplot()  
  
ggplot2::ggplot(df, aes(Y))+  
  geom_histogram(bins = 20)  
  
ggplot2::ggplot(df, aes(C2))+  
  geom_bar()
```

```
df %>%  
  select_if(is.numeric) %>%  
  cor() %>%  
  corrrplot::corrplot()
```

2 Recipes

```
recipe <- recipe(Y ~ ., data = df) %>%  
  step_...  
recipe %>% prep() %>% bake(new_data = NULL)  
recipe %>% prep() %>% juice()
```

```
step_zv()  
step_nzv()  
step_rm()  
step_filter()  
  
step_center()  
step_scale()  
step_normalize()  
step_range()
```

```
step_dummy()  
step_other()  
step_unknown()  
step_novel()
```

```
step_naomit()  
step_impute_mean()  
step_impute_median()  
step_impute_mode()  
step_impute_knn()
```

```
step_log()  
step_BoxCox()  
step_poly()  
step_interact()  
step_pca()  
step_corr()  
step_bs()
```

3 Models

Linear Regression

```
lm <- linear_reg() %>%  
  set_mode("regression") %>%  
  set_engine("lm")
```

Ridge Regression

```
ridge <- linear_reg(penalty = tune(), mixture = 0) %>%  
  set_mode("regression") %>%  
  set_engine("glmnet")
```

Lasso Regression

```
lasso <- linear_reg(penalty = tune(), mixture = 1) %>%  
  set_mode("regression") %>%  
  set_engine("glmnet")
```

Logistic Regression

```
glm <- logistic_reg() %>%  
  set_mode("classification") %>%  
  set_engine("glmnet")
```

LDA

```
lda <- discrim_linear() %>%  
  set_mode("classification") %>%  
  set_engine("MASS")
```

QDA

```
qda <- discrim_quad() %>%  
  set_mode("classification") %>%  
  set_engine("MASS")
```

Decision Tree

```
dt <- decision_tree(tree_depth = 4) %>%  
  set_engine("rpart") %>%  
  set_mode("classification")
```

Random Forest

```
rf <- rand_forest(mtry = tune(), min_n() = tune(), trees = 500) %>%  
  set_mode("classification") %>%  
  set_engine("ranger")
```

SVM

```
svm <- svm_rbf()/svm_poly()/svm_linear() %>%  
  set_mode("classification") %>%  
  set_engine("kernlab")
```

KNN

```
knn <- nearest_neighbor(neighbors = tune()) %>%  
  set_mode("classification") %>%  
  set_engine("kkn")
```

4 Workflow

```
df_wf <- workflow() %>%  
  add_recipe(recipe) %>%  
  add_model(lm)
```

4.1 Split

```
# Train/test split  
df_split <- initial_split(df, prop =  
  0.8, strata = Y)  
df_train <- training(df_split)  
df_test <- testing(df_split)  
# Cross-validation  
df_folds <- vfold_cv(df_train, v = 20,  
  strata = Y)
```

4.2 Tuning

```
# Set grid  
df_grid <- grid_regular(mtry(c(1,5)),  
  min_n(),  
  levels = 5)  
# Tuning  
doParallel::registerDoParallel()  
df_tune <- tune_grid(wf,  
  resamples =  
    df_folds,  
  grid = df_grid)
```

4.3 Fit

```
# Find the best model  
show_best(df_tune, metric = "rmse")  
df_wf <- df_wf %>%  
  finalize_workflow(select_best(df_tune,  
    metric = "rmse"))  
# Fit the model  
df_fit <- df_wf %>% last_fit(split =  
  df_split)  
df_fit %>% collect_metrics()
```

5 Metrics

```
# Classification  
conf_mat(truth = Y, estimate = .pred_class)  
roc_auc(truth = Y, estimate = .pred_class)  
roc_curve(truth = Y, estimate = .pred_class)  
acc(truth = Y, estimate = .pred_class)  
sens(truth = Y, estimate = .pred_class)  
spec(truth = Y, estimate = .pred_class)
```

```
# Regression  
rmse(truth = Y, estimate = .pred)  
rsq(truth = Y, estimate = .pred)
```

```
# Useful code  
extract_fit_parsnip()  
vip()  
tidy()  
glance()
```

Pracitcal Example 1(Regression)

```
pacman::p_load(tidymodels, tidyverse, ggplot2, vip)
df <- readRDS('./data/sample_ex.rds')

# EDA
ggplot(df, aes(Y))+
  geom_histogram()

skimr::skim(df)

# Data Cleaning
df <- df %>%
  mutate(Y= log(Y)) %>%
  mutate(X2 = ifelse(X2 < -10, NA, X2)) %>%
  mutate(X3 = as.numeric(X3)) %>%
  mutate(C1 = ifelse(C1 %in% c("a", "b", "c", "d"), C1,
    ↪ NA)) %>%
  mutate(C2 = ifelse(C2 %in% c("W", "X", "Y", "Z"), C2,
    ↪ NA))

skimr::skim(df)

mean(df$Y)
mean(df$X2, na.rm=TRUE)
mean(df$X3)

df <- df %>% na.omit()
df

# Data Splitting
set.seed(20251)
df_split <- initial_split(df, strata = Y)
df_split

df_train <- training(df_split)
df_test <- testing(df_split)

df_cv <- vfold_cv(df_train, v = 20, strata = Y)
df_cv

# Making recipe and model
df_recipe <- recipe(Y ~ ., data = df_train) %>%
  step_BoxCox(X4) %>%
  step_normalize(all_numeric_predictors()) %>%
  step_interact(~ X1:X2) %>%
  step_dummy(all_nominal_predictors())
df_recipe %>% prep() %>% bake(new_data = NULL)
```

```
lasso_reg <- linear_reg(penalty = tune(), mixture = 1) %>%
  set_engine("glmnet") %>%
  set_mode("regression")

df_wf <- workflow() %>%
  add_model(lasso_reg) %>%
  add_recipe(df_recipe)

# Model Tuning
df_grid <- grid_regular(penalty(), levels = 100)
df_grid[98,]

doParallel::registerDoParallel()
df_reg <- tune_grid(lasso_reg,
  df_recipe,
  resamples = df_cv,
  grid = df_grid)
show_best(df_reg, metric = 'rmse')

# Select the best model
best <- select_best(df_reg, metric = 'rmse')

df_wf_fin <- df_wf %>%
  finalize_workflow(best)

df_fit <- df_wf_fin %>% last_fit(df_split)
df_fit %>% collect_metrics()

df_fit %>%
  extract_fit_parsnip() %>%
  tidy()

df_model <- df_wf_fin %>%
  fit(df_train)

# Prediction
pred_data <- data.frame(
  X1 = -1,
  X2 = 0.5,
  X3 = 2,
  X4 = 1,
  C1 = factor("c", levels = c("a", "b", "c", "d")),
  C2 = factor("X", levels = c("W", "X", "Y", "Z"))
)
df_pred <- predict(df_model, pred_data)
df_pred
```